

Normalization

Advanced NLP M26

Manish Shrivastava + Aparajitha



September 3, 2026

Normalization

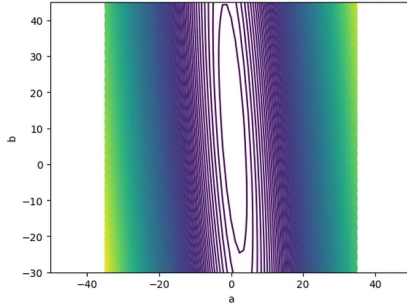
Suppose we have a simple parametric function $y = ax_1 + bx_2$ that we want to learn (say, using linear regression)

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i(a, b))^2$$



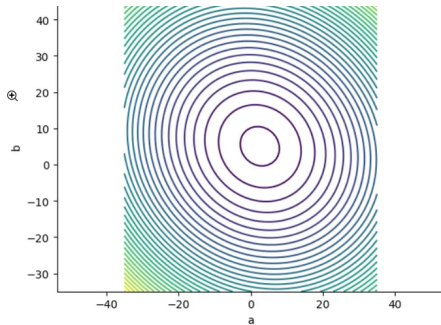
Normalization

The plot of $\mathcal{L}(a, b)$ looks like this



Normalization

We can force the distribution to be more “spread out”. Using $x_i \mapsto \frac{x_i - \mathbb{E}[x_i]}{\text{Var}(x_i)}$, we get this new loss landscape.



Why normalization?

- Increase the speed of convergence
- Reduce sensitivity to input features
- Reduce overfitting
- Help the model generalize better



BatchNorm

BatchNorm normalizes a batch of inputs $x^{(1)}, x^{(2)}, \dots, x^{(B)}$ index-wise.

$$\mu_i = \frac{1}{B} \sum_{b=1}^B x_i^{(b)}$$

$$(\sigma_i)^2 = \frac{1}{B} \sum_{b=1}^B \left(x_i^{(b)} - \mu_i \right)^2$$

$$y_i^{(b)} = \gamma_i \frac{x_i^{(b)} - \mu_i}{\sqrt{(\sigma_i)^2 + \epsilon}} + \beta_i$$

LayerNorm

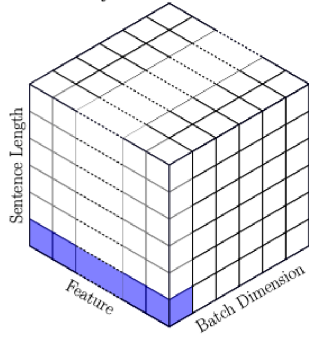
LayerNorm normalizes a batch of inputs $x^{(1)}, x^{(2)}, \dots, x^{(B)}$ input-wise.

$$\mu^{(b)} = \frac{1}{d} \sum_{i=1}^d x_i^{(b)}$$

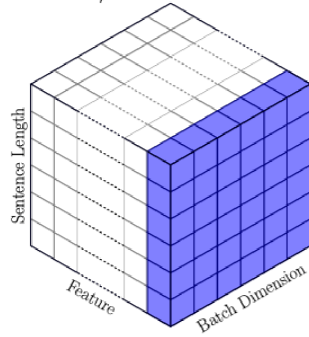
$$(\sigma^{(b)})^2 = \frac{1}{d} \sum_{i=1}^d (x_i^{(b)} - \mu^{(b)})^2$$

$$y^{(b)} = \gamma^{(b)} \frac{x^{(b)} - \mu}{\sqrt{(\sigma^{(b)})^2 + \epsilon}} + \beta^{(b)}$$

Layer Normalization



Batch/Power Normalization



RMSNorm

- A special case of LayerNorm where $\mu = 0$ is enforced.

$$y^{(b)} = \gamma^{(b)} \frac{x^{(b)}}{\sqrt{\frac{1}{d} \sum_{i=1}^d (x^{(b)})^2}} + \beta^{(b)}$$



Weight Normalization

- Separating weight vectors into magnitude and direction lets us consider direction without magnitude
- Considering both quantities as separate learnable ones speeds up convergence.
- This method is applied per neuron unlike the previous methods

$$\text{WeightNorm}(\mathbf{x}) = \frac{g}{\|\mathbf{v}\|} \mathbf{v}^\top \mathbf{x} + b$$

Residual Connections

$$x_{L+1} = \sigma(x_L + F(x_L))$$

- Used in the original transformers paper
- Well-known to mitigate vanishing gradient, since the additive gradients dominate the multiplicative gradients.



Residual Connections

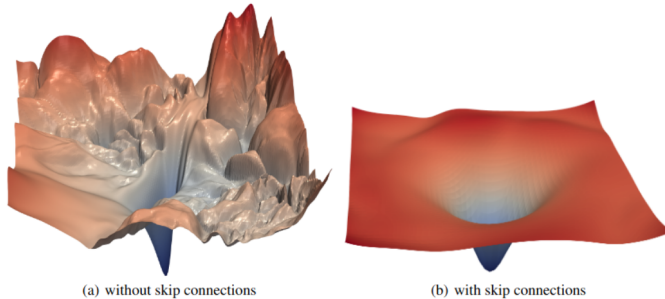
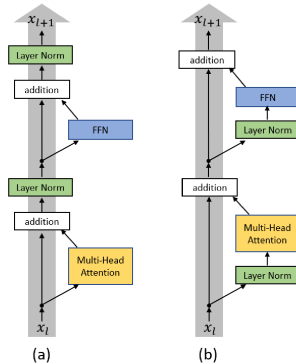


Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Pre-LN vs Post-LN



Pre-LN vs Post-LN

Tradeoffs

- Post-Norm makes training very unstable and can lead to vanishing or exploding gradients.
- When done carefully, Post-Norm doesn't result in any loss of information.
- Pre-Norm stabilizes training and avoids many gradient issues.
- Pre-Norm leads to representation collapse (each layer represents nearly the same information, measured by cosine similarity)



DeepNorm

- Adding a scaling term to the residual connection in Post-Norm
- Stabilizes gradients across really deep networks (theoretically thousands).

$$x_{L+1} = \text{LayerNorm}(\alpha x_L + F(x_L))$$

where α is a constant dependent on the total number of layers

HybridNorm

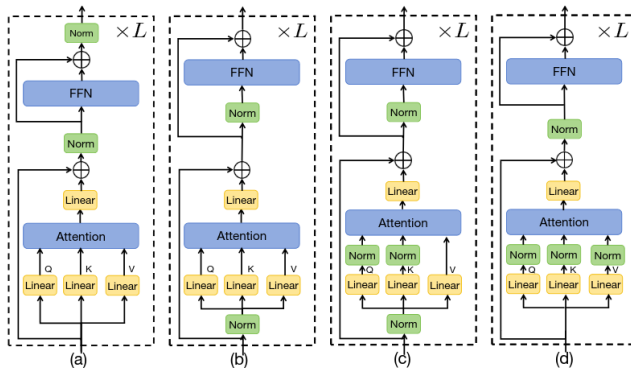


Figure 1 Illustrations of different transformer layer structures: (a) Post-Norm architecture; (b) Pre-Norm architecture; (c) Pre-Norm with QK-Norm architecture; (d) HybridNorm architecture.

HybridNorm

- Post-attention normalization is more sensitive to normalization type than other positions (empirically)
- Using LayerNorm in those places, and using RMSNorm elsewhere allows meaningful speedup (relative to only using LayerNorm) without catastrophic results (that are quite likely when only using RMSNorm).

